Author: Suhas Suresha & Dewang Sultania

SECTION 1

What a session contains

- **Session object:** A conversation bundles `session_id`, `user_id`, timestamps, and an ordered message list that must round-trip intact through storage and the Model Service.
- **Roles and content:** Messages follow the familiar role pattern (`user`, `assistant`, `system`, `tool`) with text content, matching what providers already expect for chat completion.
- **Tool turns:** A single user question can produce multiple stored messages—assistant `tool_calls`, `tool` results linked by `tool_call_id`, and the final assistant reply—so the next model call sees the full reasoning chain.
- **Timestamps:** Per-message times support pagination, auditing, and later decisions about which history to keep when budgets tighten.
- **Intentional simplicity:** A flat message list with light metadata keeps the hot path fast for read-append on every turn.

SECTION 2

The session service contract

- **Five RPCs:** GetOrCreateSession, ListSessions, AddMessages, GetMessages, and DeleteSession split lifecycle from message I/O.
- **GetOrCreateSession:** One call avoids race-prone separate create/get flows when a client may or may not already have a `session_id`.
- **Batch writes:** AddMessages accepts multiple messages atomically so tool-using turns do not partially persist.
- **Flexible reads:** GetMessages supports limit, offset, and a strategy hook for how history is assembled before it reaches the model.
- **Shared types:** Protocol buffer Message and ToolCall shapes align with the Python datamodel so servicers and SDK clients translate without semantic drift.

SECTION 3

Storage abstraction and backend

- **Pluggable persistence:** SessionStorage isolates SQL, Redis, document, or managed backends behind the same five methods the gRPC layer calls.
- **Tradeoffs by domain:** Relational stores favor durability and audit; in-memory engines favor speed and TTL; the platform does not mandate one database for every team.
- **Postgres reference:** A sessions table plus a messages table, with `JSONB` for `tool_calls`, FK cascade on delete, and indexes on `user_id` and `session_id`, illustrates production patterns.
- **Transactions:** get_or_create and add_messages use atomic commits so concurrent tabs and batched tool turns stay consistent.
- **Thin servicer:** The gRPC SessionServiceServicer receives protobuf, delegates to storage via dependency injection, and never embeds vendor-specific SQL.

SECTION 4

SDK integration

- **SessionClient:** The SDK mirrors other services with a stub, gateway metadata routing, and Python-friendly `Session` objects.
- **Developer surface:** Workflows call methods such as `get_or_create`, `add_messages`, and `get_messages` instead of hand-rolling protobuf.
- **Lazy connection:** Accessing `platform.sessions` opens the channel on first use, consistent with the Chapter 2 platform entry pattern.
- **End-to-end flow:** A typical handler fetches or creates a session, reads history, generates via the Model Service, then persists the turn before responding.

SECTION 5

Model-managed memory

- **Beyond the transcript:** Inspired by MemGPT, structured facts can live outside the verbatim log so critical details survive truncation.
- **MemoryEntry:** Key-value records scoped by `user_id` with optional `session_id` support both long-lived preferences and session-tagged provenance.
- **Storage extensions:** The same abstraction gains save, get, delete, and clear_user_memory operations; Postgres might use a dedicated memories table with JSON values.
- **Tool-driven writes:** Workflows expose a `save_memory` tool so the model decides what to persist while the platform enforces storage and audit.
- **Concurrency:** Last-write-wins, optimistic checks, or append-style merges address simultaneous updates to the same key across sessions.

SECTION 6

Managing context windows

- **Token budget:** System prompts, tools, RAG chunks, memories, and the reply reserve compete with history—only a fraction of the window is usually available for raw messages.
- **Truncation:** Keeping the most recent messages is fast and works for stateless FAQs but drops early constraints such as allergies or enterprise terms.
- **Summarization:** An LLM compresses older turns into a labeled summary block, trading verbatim detail for retained narrative at extra latency and cost.
- **Hierarchical memory:** Production stacks often combine structured memories, a rolling summary, and recent verbatim turns, ordering context for known attention biases.
- **Retrieval-augmented memory:** Cross-session questions can pull relevant past messages via the Data Service, treating prior chats as searchable episodic knowledge.
- **Strategy parameter:** Callers choose approaches through `GetMessages` strategy and budgets while the platform supplies reusable assembly building blocks.